

```

14 import static org.junit.Assert.assertEquals;
15
16 /**
17  * @author Eric Zhou
18  */
19
20 public class GlossaryTest {
21
22     /*
23      * Tests of readGlossary
24      */
25
26     /*
27      * testing readGlossary with two terms, routine test
28      */
29     @Test
30     public void readGlossary_twoterms() {
31         final String fileName = "testfile.txt";
32
33         SimpleWriter writer = new SimpleWriter1L(fileName);
34         writer.println("term1");
35         writer.println("This is a definition.");
36         writer.println("It spans two lines.");
37         writer.println();
38         writer.println("term2");
39         writer.println("Second definition.");
40         writer.println();
41         writer.close();
42
43         SimpleReader reader = new SimpleReader1L(fileName);
44         reader.close();
45
46         Map<String, String> actual = Glossary.readGlossary(fileName);
47
48         Map<String, String> expected = new Map1L<>();
49         expected.add("term1", "This is a definition. It spans two lines.");
50         expected.add("term2", "Second definition.");
51
52         assertEquals(expected, actual);
53     }
54
55     /*
56      * testing readGlossary with two line definition, boundary test
57      */
58     @Test
59     public void readGlossary_twolinedef() {
60         final String fileName = "testfile.txt";
61
62         SimpleWriter writer = new SimpleWriter1L(fileName);
63         writer.println("term1");
64         writer.println("This is a definition.");
65         writer.println("It spans two lines.");
66         writer.println();
67         writer.println("term2");
68         writer.println("Second definition.");
69         writer.println();
70         writer.close();
71
72         SimpleReader reader = new SimpleReader1L(fileName);
73         reader.close();
74
75         Map<String, String> actual = Glossary.readGlossary(fileName);
76
77         Map<String, String> expected = new Map1L<>();
78         expected.add("term1", "This is a definition. It spans two lines.");

```

```

76      expected.add("term2", "Second definition.");
77
78      assertEquals(expected, actual);
79  }
80
81  /*
82   * testing readGlossary with five terms and with one or two line definition,
83   * large number test
84   */
85  @Test
86  public void readGlossary_fiveterms() {
87      final String fileName = "testfile.txt";
88
89      SimpleWriter writer = new SimpleWriter1L(fileName);
90      writer.println("Term1");
91      writer.println("Definition of term one.");
92      writer.println();
93      writer.println("Term2");
94      writer.println("Definition of term two.");
95      writer.println("Continued on second line.");
96      writer.println();
97      writer.println("Term3");
98      writer.println("Definition three.");
99      writer.println();
100     writer.println("Term4");
101     writer.println("Fourth term definition line.");
102     writer.println();
103     writer.println("Term5");
104     writer.println("Fifth.");
105     writer.println();
106     writer.close();
107
108     Map<String, String> actual = Glossary.readGlossary(fileName);
109
110     Map<String, String> expected = new Map1L<>();
111     expected.add("Term1", "Definition of term one.");
112     expected.add("Term2", "Definition of term two. Continued on second line.");
113     expected.add("Term3", "Definition three.");
114     expected.add("Term4", "Fourth term definition line.");
115     expected.add("Term5", "Fifth.");
116
117     assertEquals(expected, actual);
118 }
119
120 /*
121  * Tests of sort
122  */
123
124 /*
125  * testing sort with a sequence needed to be sorted, routine test
126  */
127 @Test
128 public void sort_neededtobesorted() {
129     Sequence<String> seq = new Sequence1L<>();
130     seq.add(0, "banana");
131     seq.add(1, "apple");
132     seq.add(2, "grape");
133     seq.add(3, "watermelon");
134     seq.add(4, "cherry");
135
136     Glossary.sort(seq);
137
138     Sequence<String> expected = new Sequence1L<>();
139     expected.add(0, "apple");
140     expected.add(1, "banana");

```

```

144 expected.add(2, "cherry");
145 expected.add(3, "grape");
146 expected.add(4, "watermelon");
147
148 assertEquals(expected, seq);
149 }
150
151 /*
152  * testing sort with a sequence with nice order, boundary test
153  */
154 @Test
155 public void testSort_AlreadySorted() {
156     Sequence<String> seq = new Sequence1L<>();
157     seq.add(0, "apple");
158     seq.add(1, "banana");
159     seq.add(2, "cherry");
160     seq.add(3, "grape");
161     seq.add(4, "watermelon");
162
163     Glossary.sort(seq);
164
165     Sequence<String> expected = new Sequence1L<>();
166     expected.add(0, "apple");
167     expected.add(1, "banana");
168     expected.add(2, "cherry");
169     expected.add(3, "grape");
170     expected.add(4, "watermelon");
171
172     assertEquals(expected, seq);
173 }
174
175 /*
176  * testing sort with a sequence with nice order, boundary test
177  */
178 @Test
179 public void testSort_twoapple() {
180     Sequence<String> seq = new Sequence1L<>();
181     seq.add(0, "banana");
182     seq.add(1, "apple");
183     seq.add(2, "apple");
184     seq.add(3, "grape");
185
186     Glossary.sort(seq);
187
188     Sequence<String> expected = new Sequence1L<>();
189     expected.add(0, "apple");
190     expected.add(1, "apple");
191     expected.add(2, "banana");
192     expected.add(3, "grape");
193
194     assertEquals(expected, seq);
195 }
196
197 /*
198  * Tests of addLinks
199  */
200
201 /*
202  * testing addLinks with one term needed to be linked, routine test
203  */
204 @Test
205 public void addLinks_oneLink() {
206     String def = "The apple is red.";
207
208     Sequence<String> terms = new Sequence1L<>();

```

```

209        terms.add(0, "apple");
210
211        String actual = Glossary.addLinks(def, terms);
212
213        String expected = "The <a href=\"apple.html\"><b><i>apple</i></b></a> is red.";
214
215        assertEquals(expected, actual);
216    }
217
218    /*
219     * testing addLinks with two terms needed to be linked, routine test
220     */
221    @Test
222    public void addLinks_twoLinks() {
223        String def = "I like apple and banana.";
224
225        Sequence<String> terms = new Sequence1L<>();
226        terms.add(0, "apple");
227        terms.add(1, "banana");
228
229        String actual = Glossary.addLinks(def, terms);
230
231        String expected = "I like <a href=\"apple.html\"><b><i>apple</i></b></a> "
232            + "and <a href=\"banana.html\"><b><i>banana</i></b></a>.";
233
234        assertEquals(expected, actual);
235    }
236
237    /*
238     * testing addLinks with no term needed to be linked, boundary test
239     */
240    @Test
241    public void addLinks_noLink() {
242        String def = "The sky is blue.";
243        Sequence<String> terms = new Sequence1L<>();
244
245        String actual = Glossary.addLinks(def, terms);
246        String expected = "The sky is blue.";
247
248        assertEquals(expected, actual);
249    }
250
251    /*
252     * Tests of nextWordOrSeparator
253     */
254
255    /*
256     * testing nextWordOrSeparator with start with word, boundary test
257     */
258    @Test
259    public void nextWordOrSeparator_Word() {
260        String text = "apple pie!";
261        int position = 0;
262
263        Set<Character> separators = new Set1L<>();
264        separators.add(' ');
265        separators.add('!');
266
267        String actual = Glossary.nextWordOrSeparator(text, position, separators);
268        String expected = "apple";
269
270        assertEquals(expected, actual);
271    }
272
273    /*

```

```

274 * testing nextWordOrSeparator with start with separator, boundary test
275 */
276 @Test
277 public void nextWordOrSeparator_Separator() {
278     String text = "hello world";
279     int position = 5;
280
281     Set<Character> separators = new Set1L<>();
282     separators.add(' ');
283
284     String actual = Glossary.nextWordOrSeparator(text, position, separators);
285     String expected = " ";
286
287     assertEquals(expected, actual);
288 }
289
290 /*
291 * testing nextWordOrSeparator with start with word in middle, routine test
292 */
293 @Test
294 public void nextWordOrSeparator_Middle() {
295     String text = "Hi,there!";
296     int position = 3;
297
298     Set<Character> separators = new Set1L<>();
299     separators.add(' ');
300     separators.add(',');
301     separators.add('!');
302
303     String actual = Glossary.nextWordOrSeparator(text, position, separators);
304     String expected = "there";
305
306     assertEquals(expected, actual);
307 }
308
309 /*
310 * Tests of writeIndexPage
311 */
312
313 /*
314 * testing writeIndexPage with two terms, routine test
315 */
316 @Test
317 public void testWriteIndexPage_twoterms() {
318     final String folder = "test";
319     final String filePath = folder + "/index.html";
320
321     Sequence<String> terms = new Sequence1L<>();
322     terms.add(0, "apple");
323     terms.add(1, "banana");
324
325     Glossary.writeIndexPage(terms, folder);
326
327     SimpleReader in = new SimpleReader1L(filePath);
328     assertEquals("<html><head><title>Glossary</title></head><body>", in.nextLine());
329     assertEquals("<h2>Glossary Index</h2>", in.nextLine());
330     assertEquals("<ul>", in.nextLine());
331     assertEquals("<li><a href=\"apple.html\">apple</a></li>", in.nextLine());
332     assertEquals("<li><a href=\"banana.html\">banana</a></li>", in.nextLine());
333     assertEquals("</ul>", in.nextLine());
334     assertEquals("</body></html>", in.nextLine());
335     in.close();
336 }
337
338 /*

```

```

3396 * testing writeIndexPage with three terms, routine test
340 **/
341 @Test
342 public void testWriteIndexPage_threeterms() {
343     final String folder = "test";
344     final String filePath = folder + "/index.html";
345
346     Sequence<String> terms = new Sequence1L<>();
347     terms.add(0, "apple");
348     terms.add(1, "banana");
349     terms.add(2, "cherry");
350
351     Glossary.writeIndexPage(terms, folder);
352
353     SimpleReader in = new SimpleReader1L(filePath);
354     assertEquals("<html><head><title>Glossary</title></head><body>", in.nextLine());
355     assertEquals("<h2>Glossary Index</h2>", in.nextLine());
356     assertEquals("<ul>", in.nextLine());
357     assertEquals("<li><a href=\"apple.html\">apple</a></li>", in.nextLine());
358     assertEquals("<li><a href=\"banana.html\">banana</a></li>", in.nextLine());
359     assertEquals("<li><a href=\"cherry.html\">cherry</a></li>", in.nextLine());
360     assertEquals("</ul>", in.nextLine());
361     assertEquals("</body></html>", in.nextLine());
362     in.close();
363 }
364
365 /*
366  * Tests of writeTermPage
367  */
368
369 /*
370  * testing writeTermPage with apple, routine test
371  */
372 @Test
373 public void writeTermPage_apple() {
374     final String folder = "test";
375     final String term = "apple";
376     final String filePath = folder + "/" + term + ".html";
377
378     String def = "apple is red.";
379
380     Sequence<String> terms = new Sequence1L<>();
381     terms.add(0, "apple");
382
383     Glossary.writeTermPage(term, def, terms, folder);
384
385     SimpleReader in = new SimpleReader1L(filePath);
386
387     assertEquals("<html><head><title>apple</title></head><body>", in.nextLine());
388     assertEquals("<h2><b><i><span style=\"color:red\">apple</span></i></b></h2>",
389         in.nextLine());
390     assertEquals("<p><a href=\"apple.html\"><b><i>apple</i></b></a> is red.</p>",
391         in.nextLine());
392     assertEquals("<p><a href=\"index.html\">Back to index</a></p>", in.nextLine());
393     assertEquals("</body></html>", in.nextLine());
394
395     in.close();
396 }
397
398 /*
399  * testing writeTermPage with banana, routine test
400  */
401 @Test
402 public void testWriteTermPage_Banana() {
403     final String folder = "test";

```

```

4046    final String term = "banana";
405    final String filePath = folder + "/" + term + ".html";
406
407    String def = "banana is yellow";
408
409    Sequence<String> terms = new Sequence1L<>();
410    terms.add(0, "banana");
411
412    Glossary.writeTermPage(term, def, terms, folder);
413
414    SimpleReader in = new SimpleReader1L(filePath);
415
416    assertEquals("<html><head><title>banana</title></head><body>", in.nextLine());
417    assertEquals("<h2><b><i><span style=\"color:red\">banana</span></i></b></h2>",
418        in.nextLine());
419    assertEquals("<p><a href=\"banana.html\"><b><i>banana</i></b></a> is yellow</p>",
420        in.nextLine());
421    assertEquals("<p><a href=\"index.html\">Back to index</a></p>", in.nextLine());
422    assertEquals("</body></html>", in.nextLine());
423
424    in.close();
425 }
426
427 }
428

```